AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Dictionaries and records

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Name the key. Inspect the value. Check the record contract.

Read and update dictionary records without erasing legitimate zero or inventing missing data.

Keep one habit: Blue/Red record map → zero/membership trace → six schema/update cases → selected count function → portfolio evidence.

MAIN LESSON · SECTION 1

Which named field exists, and what value does it hold?

> Separate field identity, stored zero and unavailable data.

Your learning goals

I can read and update named **dictionary** fields.

I can connect a **record** to a function call.

I can distinguish missing fields from zero values.

Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from Return values and local state. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Lists and indexing. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Boolean conditions. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Read and update named **dictionary** fields
- Connect a **record** to a function call
- Distinguish missing fields from zero values
- What does remaining(8, 3) return when its body returns capacity - used?

> **My starting explanation**

Your end goal

Trace dictionaries and **record**s and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Give each count a named field

> A readable field name helps interpretation but does not validate the record.

Read the cupboard schema

A fictional cupboard **record** has a name, capacity and used count. A list asks us to remember positions. Named fields make the intended meaning visible.

Our schema requires name as text and capacity and used as nonnegative integers, with used no greater than capacity.

Understand key/value and schema roles

A dictionary maps keys to values. In a record, a named key such as "used" states what its value means, instead of relying on a list position. The dictionary can represent a schema-breaking record just as easily as a valid one. Keep the task’s field types, count constraints and missing-data rules separate from successful lookup or arithmetic.

Your first explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Read fields and update one named value

> The complete Blue and Red inputs are labelled before comparing them.

Complete main dictionary-to-function script

```
cupboard = {"name": "Blue", "capacity": 12, "used": 4}
def remaining(capacity, used):
    return capacity - used

print(remaining(cupboard["capacity"], cupboard["used"]))
cupboard["used"] = 6
print(remaining(cupboard["capacity"], cupboard["used"]))
```

Trace Blue read and update

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Read used	4	12 - 4 = 8
Update used	6	12 - 6 = 6

Read each row from left to right and connect the evidence to the stated rule.

Keep the practice records distinct

Separate complete fictional record	Initial data and intended use
Blue · Main script	{"name":"Blue","capacity":12,"used":4}; update used to 6, remaining 8 then 6
Red · W1/W2 and error practice	{"name":"Red","capacity":9,"used":2}; W2 changes used to 9, remaining 0
Green · Earlier additional transfer	Its own task supplies capacity 15/used 6 then used 10; keep separate
Schema	name text, capacity/used nonnegative integers and used<=capacity. Example arithmetic does not enforce it.

Read lookup and schema limitations

Square brackets look up the value for a key. The quoted key "used" is text; its value is an integer. Assigning to an existing key replaces that field. Other fields remain the same.

Dictionary syntax does not enforce our schema: the programmer must check it.

Explain the deciding check

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare a present zero with a missing field

> A successful arithmetic result can still violate the declared record schema.

Predict the supported change

- Change the Red **record** used to 9. State remaining spaces and whether capacity changes.

Keep the procedure fixed. Explain what changed in the inputs and result.

Predict the pictured variant

- Change: Add the field price with value 0.

▼ Check after predicting

Prediction: The price lookup now returns 0 instead of None.

Reason: A **record**ed zero is a present value, whereas the earlier lookup had no field.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace zero, absent lookup and key membership

> Field existence and truthiness of the value are different questions.

Read the complete shelf record and policy scope

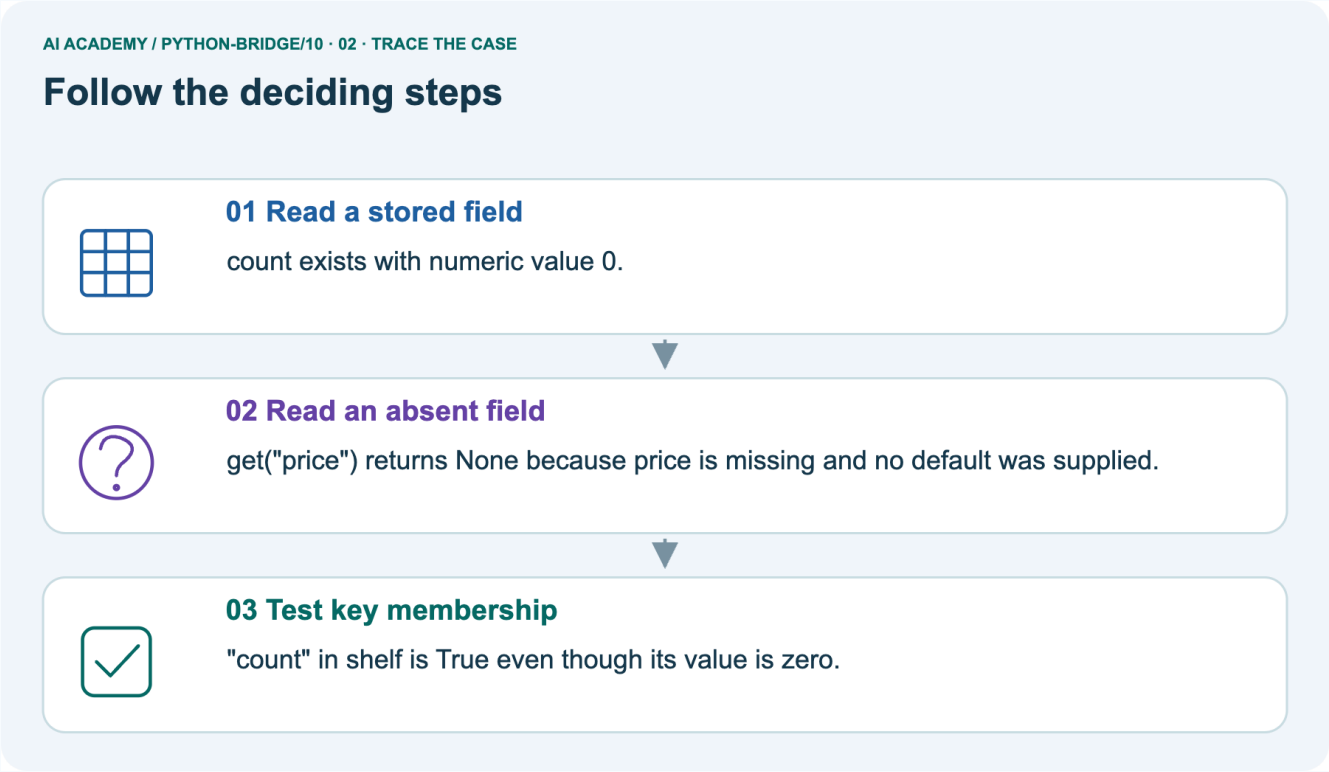
A zero field is still present. A **dictionary record** describes a shelf. The program distinguishes missing data from an actual zero.

Python 3: shelf={"count":0,"name":"Atlas"}; print(shelf["count"]); print(shelf.get("price")); print("count" in shelf).

For the pictured shelf, count is present with 0 and price is absent. get without a default returns None for absence, while membership checks whether the key exists. A present key can also explicitly hold None: get then gives None but membership is True. Do not call get alone a presence test.

The selected describe function uses its own narrow policy that absent/None count is unavailable and returns the label missing; supplied numeric 0 must remain a value. It does not validate arbitrary record types or all possible counts.

Trace the three dictionary operations



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Read a stored field	count exists with numeric value 0.
2. Read an absent field	get("price") returns None because price is missing and no default was supplied.
3. Test key membership	"count" in shelf is True even though its value is zero.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Dictionary keys name fields. Presence is different from truthiness of a value; a measured zero should not be discarded as if the field were absent.

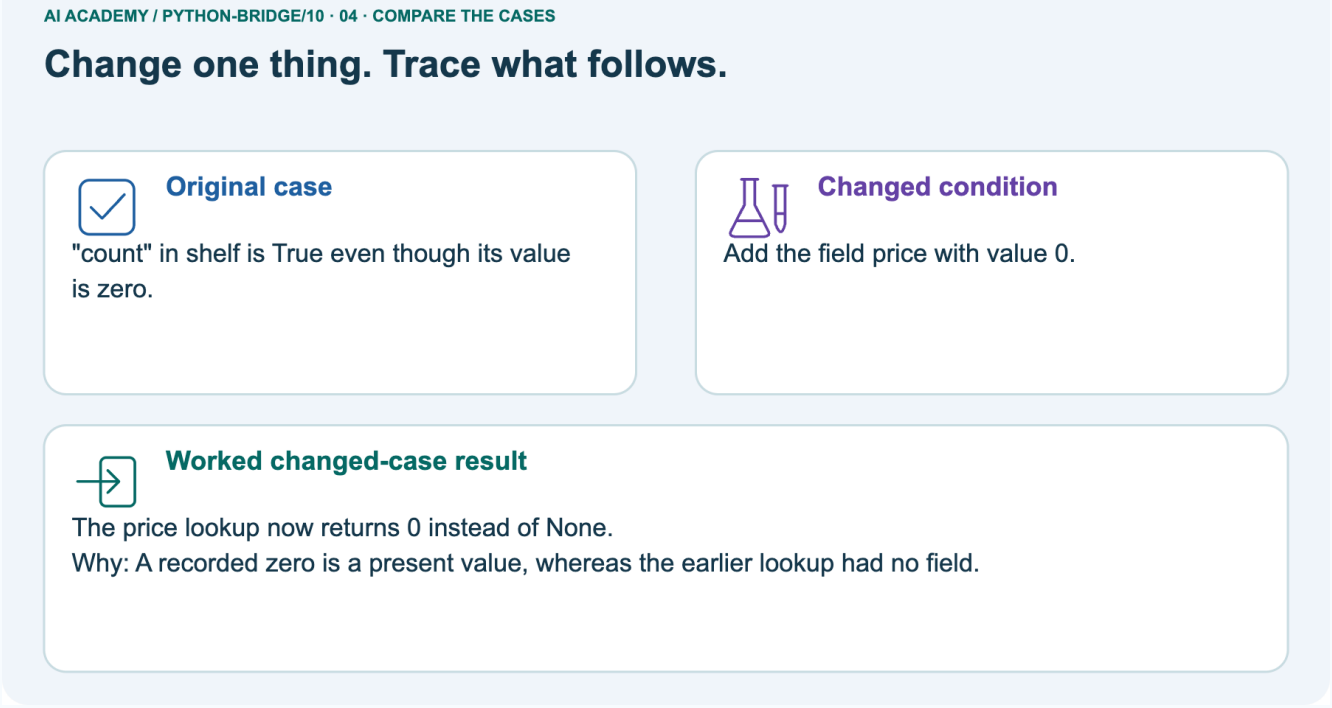
Complete runnable shelf program

```
shelf = {"count": 0, "name": "Atlas"}
print(shelf["count"])
print(shelf.get("price"))
print("count" in shelf)
```

Predict adding price with value 0

- Change one thing: Add the field price with value 0. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

State explicit None and schema limits

- None is also a possible explicitly stored value; real **record** contracts must define how missing and null values are represented.

- Read the labelled picture: Predict all three original **output**s. Explain the difference between count=0 and missing price, then predict the lookup after price=0 is added.

▼ Earlier wording, alternatives and picture transcripts

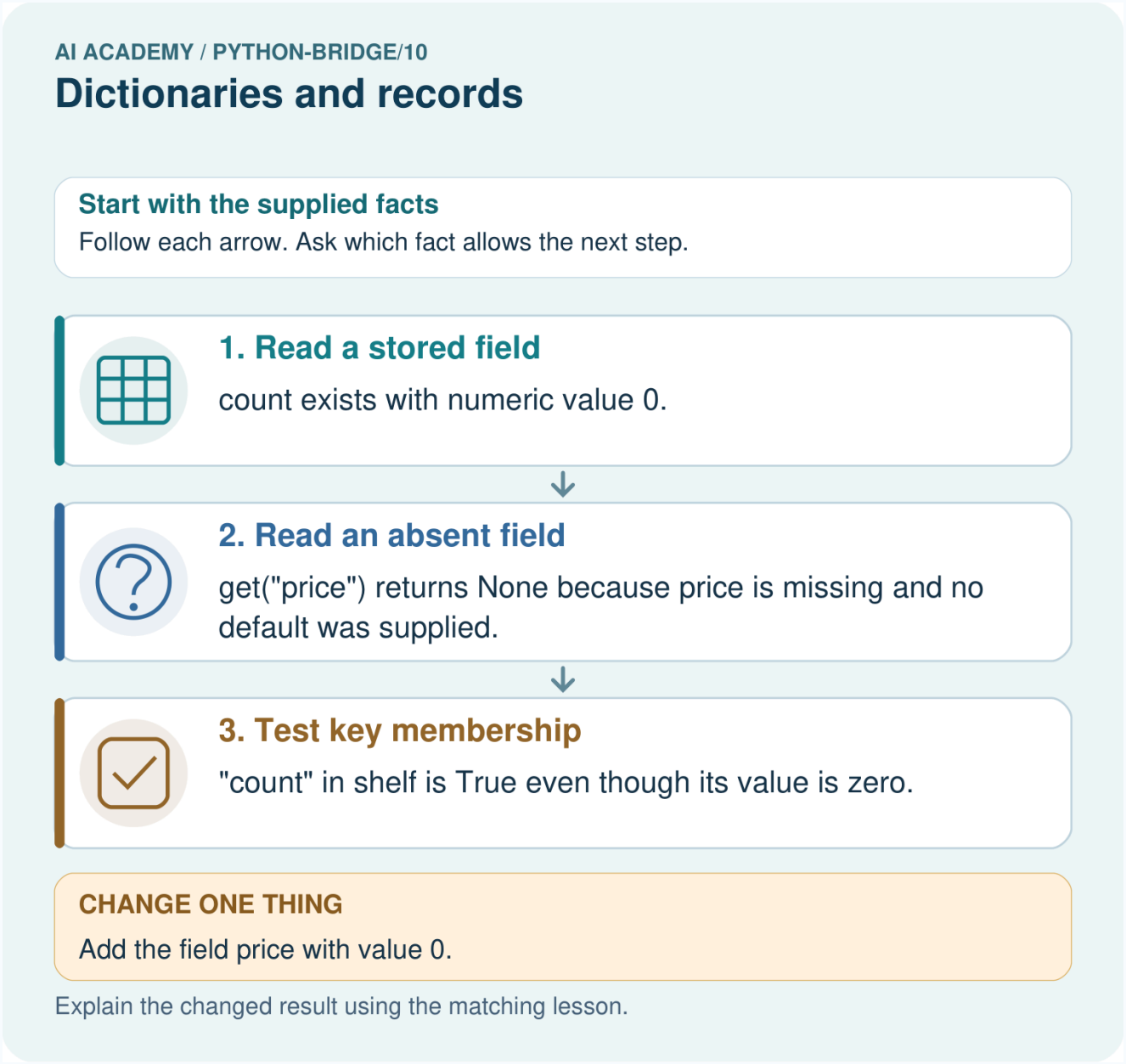
> **Set used to 12. Predict the returned spaces. Does a used value of 0 mean the field is missing?**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Dictionaries and records

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find the wrong-case key without inventing a new field

> Correct the intended lookup rather than adding an unrelated key.

Read the Used-versus-used error

cupboard["Used"] raises **KeyError** here because keys are case sensitive. Correct it to "used". Adding a new field called "Used" would create a second key and leave the original unchanged; it would not repair the intended lookup.

Diagnose and repair the Red-record lookup

- The code reads **record**["Used"] from the Red **record**. Explain the error and repair it without adding a field.

> **State one repair and the checks you would rerun.**

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise the complete Red update with less help

> Use Red capacity 9 rather than carrying Blue capacity 12 into the task.

Read and explain Red fields

- For {"name": "Red", "capacity": 9, "used": 2}, state the keys, value of used and remaining spaces.

> **Set used to 12. Predict the returned spaces. Does a used value of 0 mean the field is missing?**

> **Explain your prediction to a partner before comparing results.**

Complete W2 Red update

Use workbook W2 for the connected example “Dictionaries and **record**s”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Audit run-success-as-validity reasoning

- Use the worked case for Dictionaries and **record**s. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured zero/missing contrast

- Predict all three original **output**s. Explain the difference between count=0 and missing price, then predict the lookup after price=0 is added.

> **My explanation using a pictured step and the given facts**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Reserve selected work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace an update

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Check six complete record, update and missing-data cases

> Named fields improve interpretation; validation and source accuracy still need their own checks.

All supplied record cases

Complete case	Expected evidence
A · Red initial 9 capacity/2 used	Keys name/capacity/used; used 2, remaining 7
B · Correct W2 sets Red used 9	Remaining 0; capacity still 9
C · Read Red["Used"] with only lowercase used supplied	KeyError; fix the lookup to lowercase, no new key needed
D · {"name":"Test","capacity":3,"used":4}	Arithmetic -1 is representable, but violates used<=capacity schema
E · Shelf count 0, price absent	count membership True, value 0; price get None
F · Add price 0, then compare explicit price None	Both keys are present; values differ. None is not a recorded zero.

Complete Red W2 update program

```
record = {"name": "Red", "capacity": 9, "used": 2}
def remaining(capacity, used):
    return capacity - used

record["used"] = 9
print(remaining(record["capacity"], record["used"]))
print(record["capacity"])
```

Predict, run or trace and preserve the schema boundary

1. Use the complete Red input and W2 script, then trace C–F one at a time from their stated initial records. Keep Blue's earlier update separate. For C, preserve the KeyError and correct only key spelling. For D, reject the schema-breaking record even though subtraction finishes.

   For E/F, inspect both key membership and lookup value; a missing key and an explicit None can share a get result without being the same key-presence state. Never default missing measurements to zero to make arithmetic run. Record predicted, executed or paper-traced evidence. Keep the selected describe-count function unused until the individual attempt.

Keep the evidence record

Change the Red **record** used to 9. State remaining spaces and whether capacity changes.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or "not run"	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep "not run" if you only traced the procedure.

▼ Reference · Original Blue program

```
cupboard = {"name": "Blue", "capacity": 12, "used": 4}
def remaining(capacity, used):
    return capacity - used
```

```
print(remaining(cupboard["capacity"], cupboard["used"]))
cupboard["used"] = 6
print(remaining(cupboard["capacity"], cupboard["used"]))
```

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the selected count-record function

> Zero is supplied data even though its Boolean truth test is False.

Selected complete task and missing-data policy

Predict the three **output**s and trace the named-field update. Explain why zero and a missing count produce different results. If the condition becomes if not count, which supplied **record** is first handled incorrectly?

```
def describe(record):
    count = record.get("count")
    if count is None:
        return "missing"
    return count
record = {"count": 0}
print(describe(record))
record["count"] = 3
print(describe(record))
print(describe({}))
```

Use the selected function's supplied records 0, updated 3 and absent field. Its None check handles unavailable data under this toy policy; it is not a general key-membership or type validator. Trace the changed if not count condition separately and identify the first supplied value it misclassifies.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. **Record** any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. **Record** whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Additional practice · Earlier Green record transfer**

Create a Green **record** with capacity 15 and used 6. Pass its fields to remaining and store the result. Update used to 10 and repeat. Predict both results.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Read, update and pass a dictionary record

Attempt this without hints

Use feedback to improve

MAIN LESSON · SECTION 11

Keep keys, values and the record schema on the portfolio sheet

> A field update changes stored data, not the world it describes.

This week's contribution

Add a labelled dictionaries and **record**s **evidence** sheet to Python readiness portfolio. Show how you can read and update named **dictionary** fields.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this **record** to last week's **record**; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain key presence, lookup value and schema validity

> A record can be readable, populated and still invalid for the task.

Explain without notes

- Close your notes. Explain this goal in your own words: Read and update named **dictionary** fields

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
Dictionary	A mapping from unique keys to values
Key	A name used to look up a value
Record	Related fields describing one item
KeyError	The error from looking up an absent dictionary key

Use two terms to explain a step in this week's example.

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- For {"name": "Red", "capacity": 9, "used": 2}, state the keys, value of used and remaining spaces.
- The code reads **record**["Used"] from the Red **record**. Explain the error and repair it without adding a field.
- A **record** has capacity 3 and used 4. It gives -1 spaces without an error. Should it be accepted? Explain.

Explain which evidence supports your conclusion and which claim remains unproved.

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- Named fields reduce position mistakes but do not prove that a **record** is accurate. A capacity of 2 with used of 5 is representable in a **dictionary** and violates this schema.

- A **record** has capacity 3 and used 4. It gives -1 spaces without an error. Should it be accepted? Explain.

> **A record has capacity 3 and used 4. It gives -1 spaces without an error. Should it be accepted? Explain.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Read used	4	12 - 4 = 8
Update used	6	12 - 6 = 6

Read each row from left to right and connect the evidence to the stated rule.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Read and update named dictionary fields	Explain the deciding step, show a trace or test, and state one limit.

By the end, I can	How I will show it
Connect a record to a function call	Explain the deciding step, show a trace or test, and state one limit.
Distinguish missing fields from zero values	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ **Earlier wording, alternatives and picture transcripts**

Start with this question: A zero field is still present. A **dictionary record** describes a shelf. The program distinguishes missing data from an actual zero.

▼ **Extra practice · Predict, check and repair**

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- A zero field is still present. A **dictionary record** describes a shelf. The program distinguishes missing data from an actual zero.
- Which labelled step matches this detail? "count" in shelf is True even though its value is zero.
- 1. Test key membership

- 2. Read an absent field
- 3. Read a stored field
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or **record**s it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Test key membership "count" in shelf is True even though its value is zero. **Dictionary** keys name fields.

Presence is different from truthiness of a value; a measured zero should not be discarded as if the field were absent.

Changed-case question and hint

- Changed-case practice: Add the field price with value 0. Which conclusion is supported by the supplied facts?
- 1. The price lookup now returns 0 instead of None.
- 2. We can decide the result without examining the changed **input** or the procedure.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: The price lookup now returns 0 instead of None. A **record**ed zero is a present value, whereas the earlier lookup had no field.

Repair and keep the record

Review Return values and local state (python-bridge/9). Goal: Use a returned value in a later calculation. Short practice: Trace the two returned values and explain why first stays seven.

What happens if the caller tries to print result directly?. Return to this week and explain what you changed.

Review Lists and indexing (python-bridge/7). Goal: Distinguish a list index from the value stored at that position. Short practice: Label the three index positions.

Predict all original **output**s, then name the first **output** and stopping point after using index 3.. Return to this week and explain what you changed.

Review Boolean conditions (python-bridge/4). Goal: Translate stated requirements into comparisons. Short practice: Give both comparison results for ages 11, 15 and 16.

Which comparison rejects each outside value, and why does and matter?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

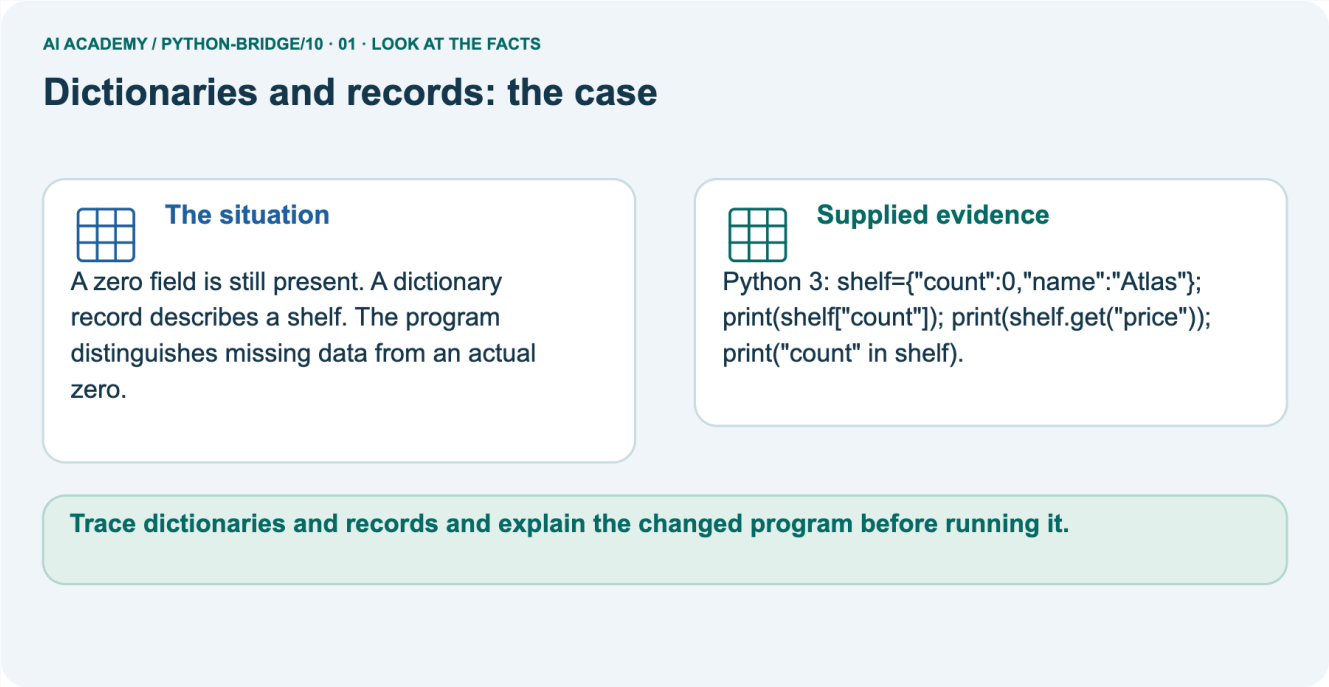
▼ Reference · The six-picture journey

REFERENCE · SECTION 18

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. Dictionaries and records: the case



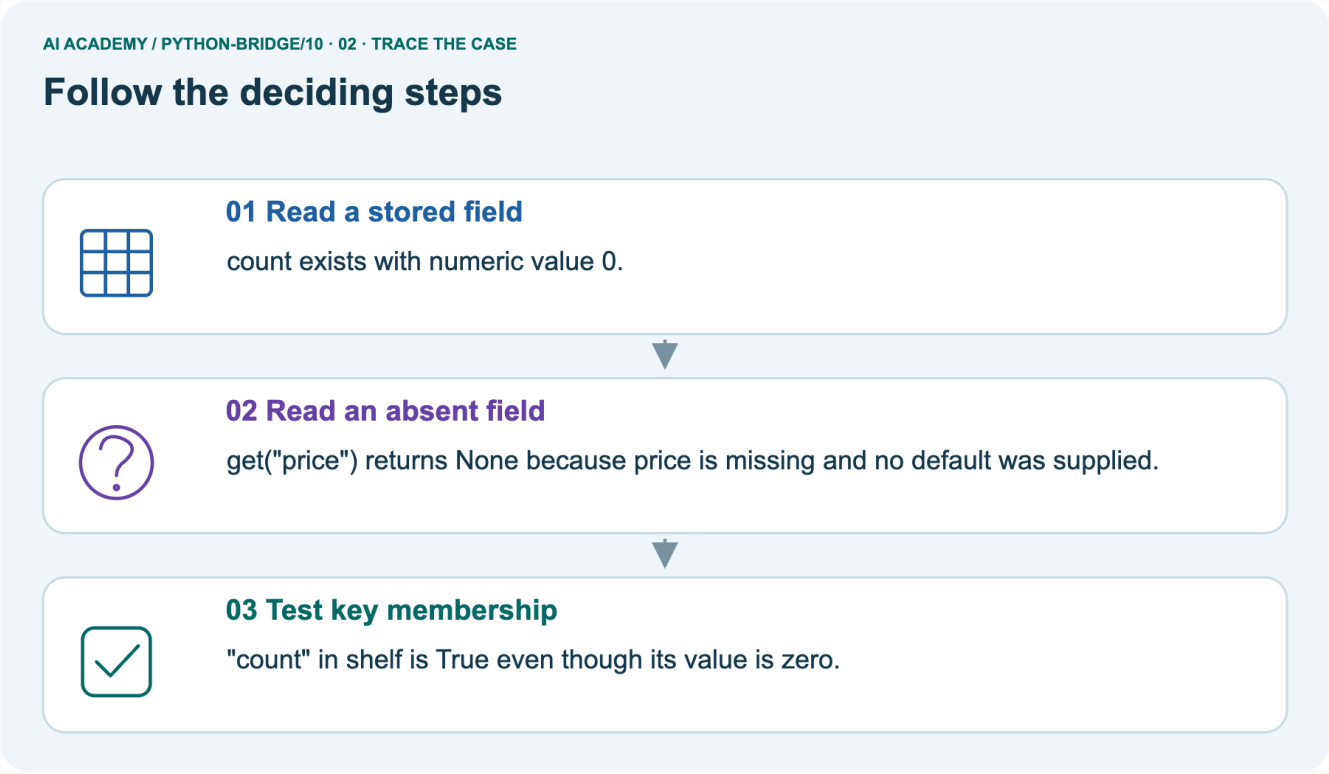
Follow the labelled picture with the matching explanation. Open the vector diagram

A zero field is still present. A **dictionary record** describes a shelf. The program distinguishes missing data from an actual zero.

Supplied facts: Python 3: shelf={"count":0,"name":"Atlas"}; print(shelf["count"]); print(shelf.get("price")); print("count" in shelf).

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

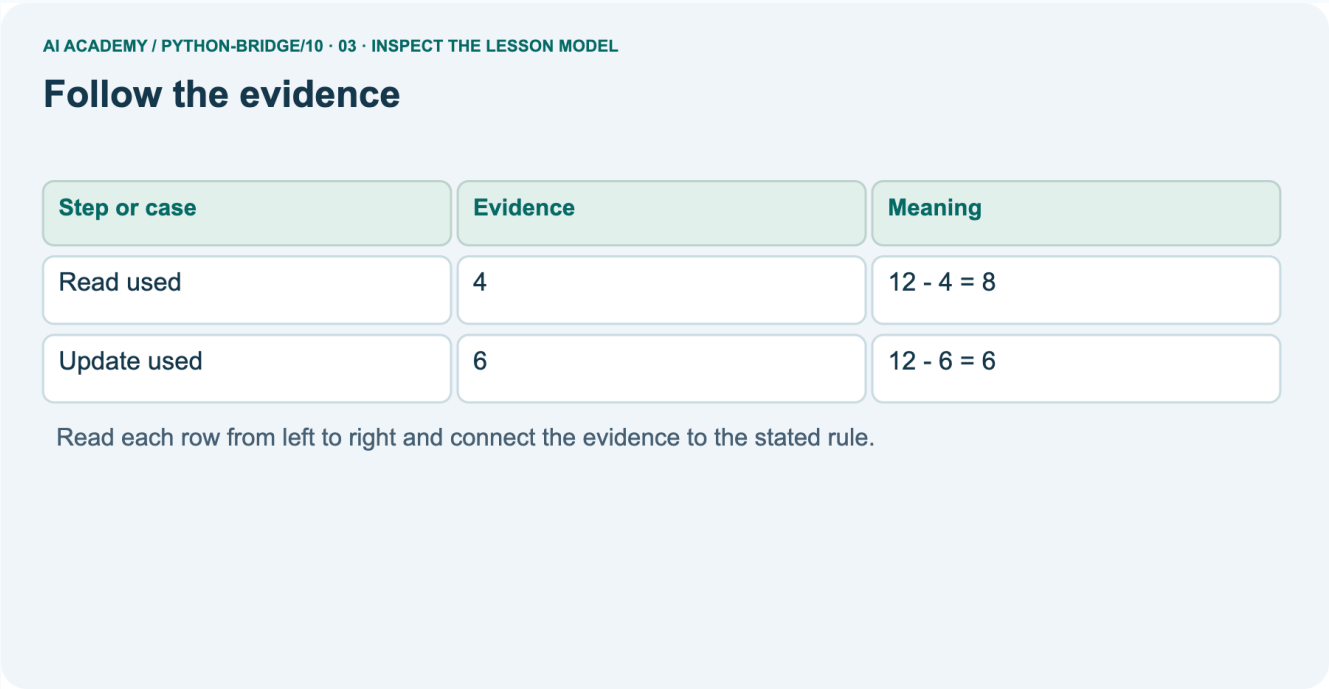


Follow the labelled picture with the matching explanation. Open the vector diagram

Read a stored field: count exists with numeric value 0. → Read an absent field: get("price") returns None because price is missing and no default was supplied. → Test key membership: "count" in shelf is True even though its value is zero.

Point to each numbered stage and explain which **record**ed **input** it uses.

03. Follow the evidence

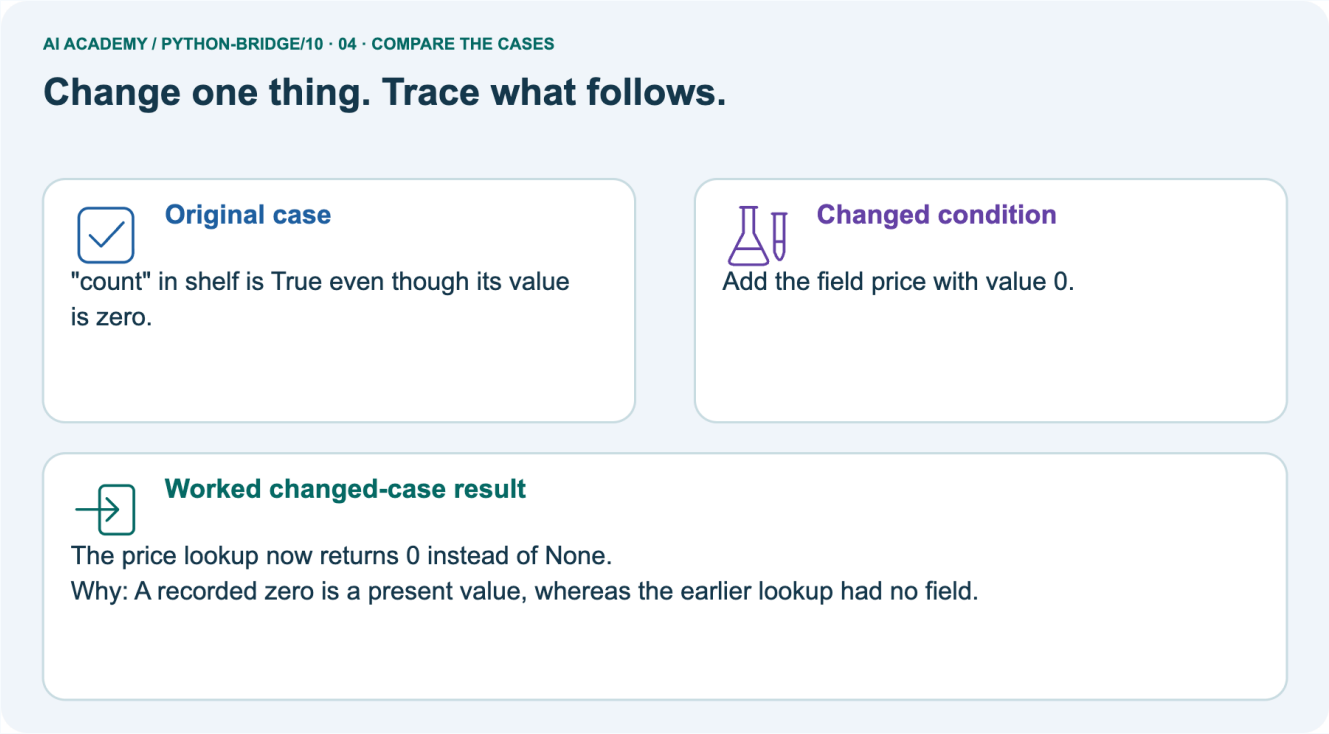


Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. Read used | 4 | 12 - 4 = 8. Update used | 6 | 12 - 6 = 6

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



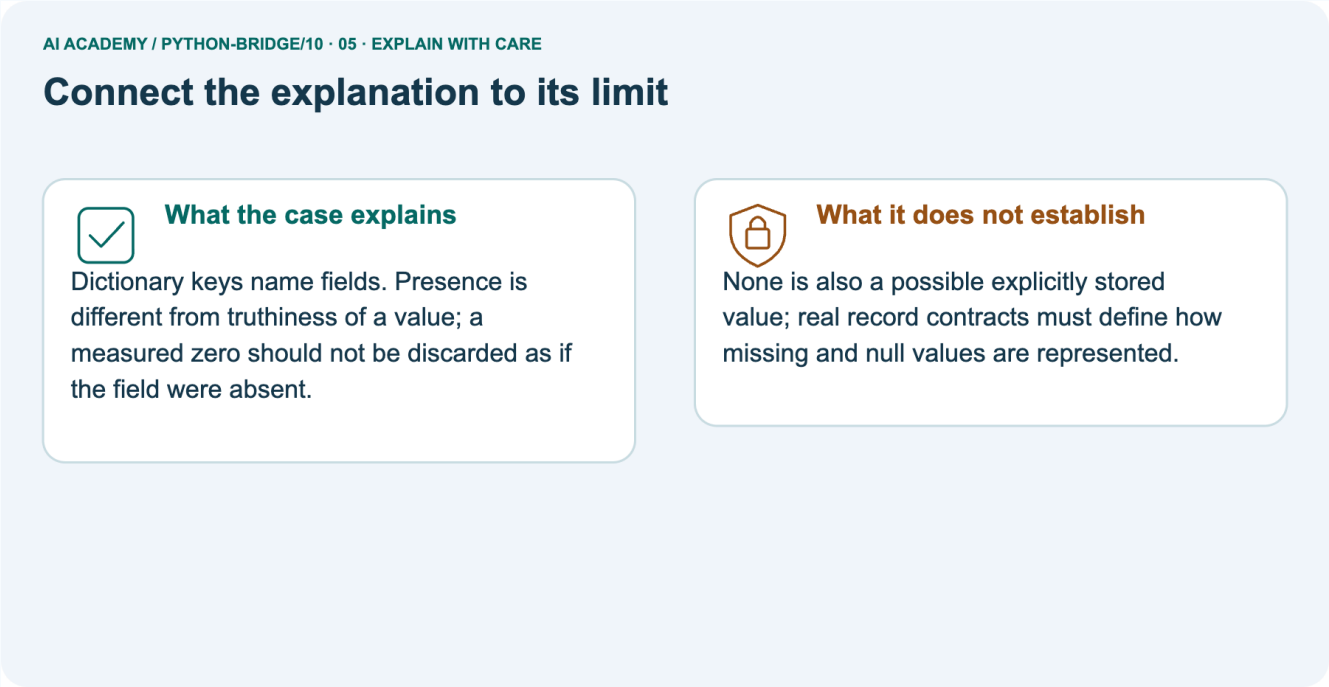
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: "count" in shelf is True even though its value is zero. Changed condition: Add the field price with value 0. Expected result: The price lookup now returns 0 instead of None.

Reason: A **record**ed zero is a present value, whereas the earlier lookup had no field.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



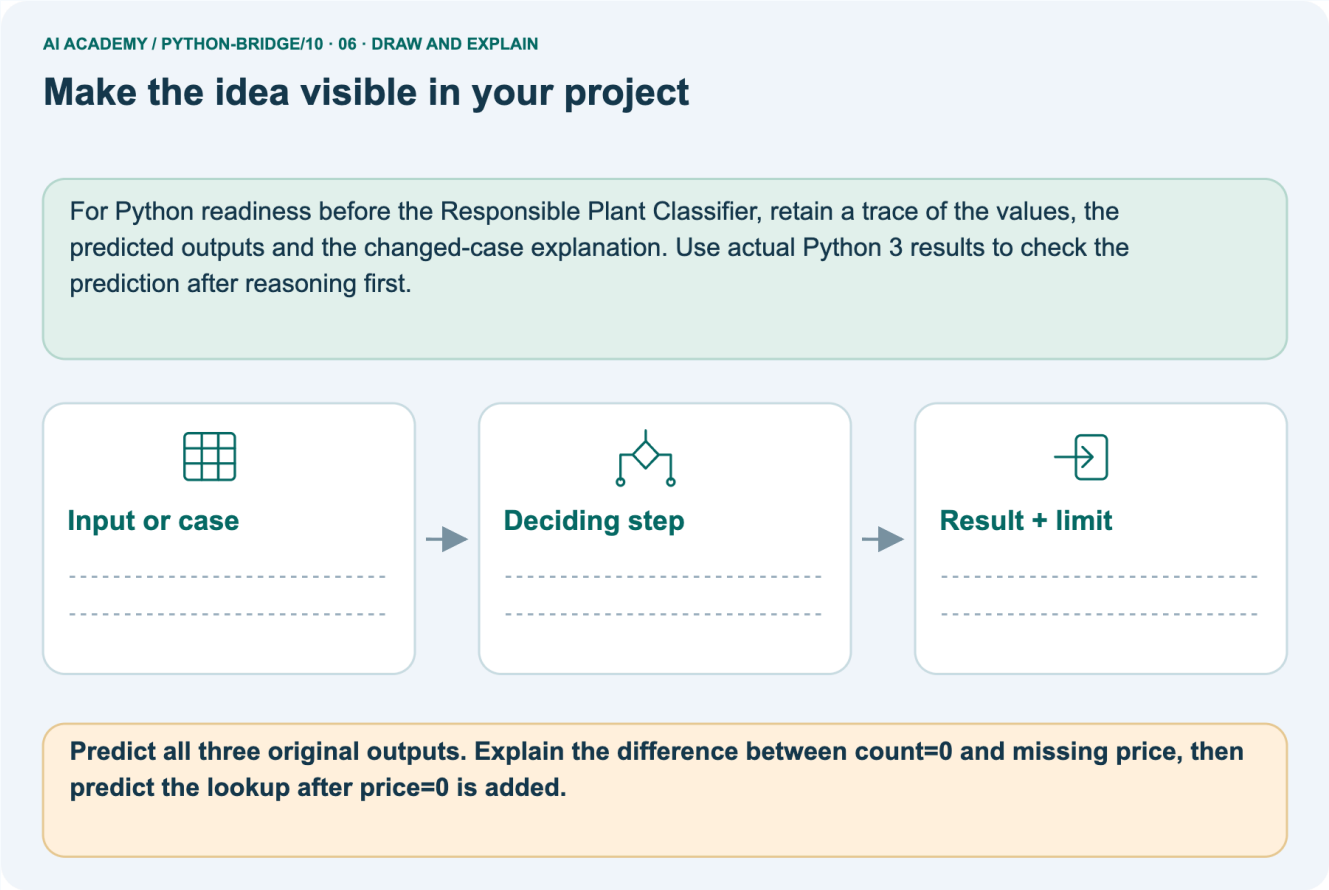
Follow the labelled picture with the matching explanation. Open the vector diagram

Dictionary keys name fields. Presence is different from truthiness of a value; a measured zero should not be discarded as if the field were absent.

Limit: None is also a possible explicitly stored value; real **record** contracts must define how missing and null values are represented.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Predict all three original **output**s. Explain the difference between count=0 and missing price, then predict the lookup after price=0 is added.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Dictionaries and records: the case

02. Follow the deciding steps

03. Follow the evidence

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.